

信息与通信工程学院
信息理论与技术教研中心



通信网基础

第五章

网络拓扑结构分析



先进网络技术实验室
望育梅

ymang@bupt.edu.cn
主楼1008

概述



- 网络拓扑结构分析是很基本，也是很重要的问题。
- 拓扑结构是通信网规划和设计的第一层次问题。
- 通信网的拓扑结构可以用图论的模型来代表，本章分析的主要问题为最小支撑树、最短路径和网络流量安排等问题。



主要内容

- 5.1 图论基础

5.1.1 图的定义和基本概念

5.1.2 树

5.1.3 割集

5.1.4 图的矩阵表示

- 5.2 最短路径问题
- 5.3 网络流量问题
- 5.4* 双权问题

图论基础



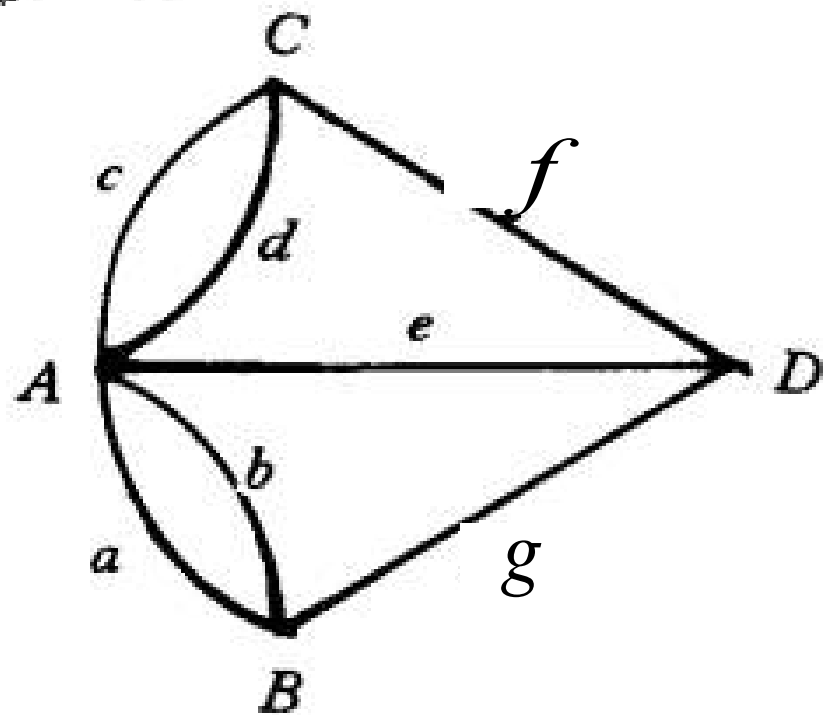
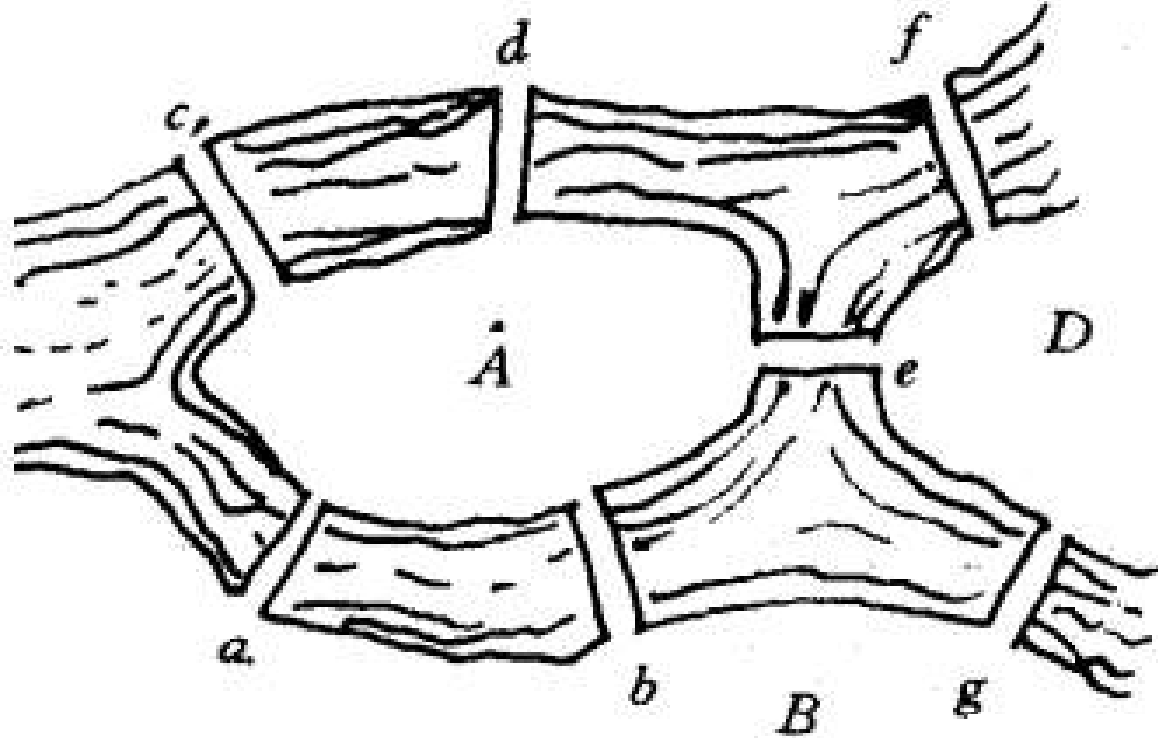
- 图论基础，即：图的定义和基本概念、树、割集。
- 重点：
 - 理解图的相关定义和树、支撑树、割集等概念；
 - 掌握连通、基本割集、基本圈的分析 and 判断。
- 难点：
 - 树的等价定义的理解和应用；
 - 割边集、割端集、反圈等定义的理解。

5.1 图论基础



5.1.1 图的定义和基本概念

- 图论是应用数学的一个分支，有着丰富的内容，本节介绍它的一些概念和结论。
- 例5.1 欧拉(Euler) 7桥问题。
有一些河穿过哥尼斯堡城，将该城分为4个部分，有7座桥将城中各个部分相连，当地有一个游戏，问能否从城的某一个部分出发遍历每一座桥同时不重复经历任何一座桥。





5.1.1 图的定义和基本概念

- 解：Euler分析了这个问题，并且用图5.1来代表这个问题，4个端点表示4个城区，而边表示7座桥。如果定义端点的度为和它关联的边的个数，那么在图5.1中，各端点的度为3, 3, 3, 5。
- 如果存在一个遍历每座桥的漫游，除去起点和终点，对每个中间端点总是一进一出，所以那些中间端点的度应该为偶数，由于图5.1中度为奇数的端大于2个，这样图5.1不可能存在一个遍历所有桥的漫游。

图的定义

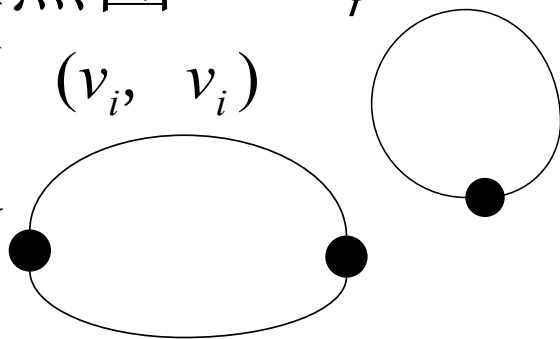


- **定义5.1** 所谓一个图 G ，是指给了一个端点集合 V ，以及边的集合或 V 中元素的序对集合 E ，图一般用 $G = (V, E)$ 来表示。
- 如果图 G 有 n 端 m 条边，可将 V 和 E 表示为 $V = \{v_1, v_2, \dots, v_n\}$, $E = \{e_1, e_2, \dots, e_m\}$ 。
- 某边的端为 v_i, v_j ，称这边和端 v_i, v_j 关联，这个边也可记为： (v_i, v_j) 或 $e_{i,j}$ 。



一些基本概念

- 有限图 $|V|, |E| < \infty$
- 无向图 $(v_i, v_j) \Leftrightarrow (v_j, v_i)$
- 有向图
- 空图 $V = \phi$
- 孤立点图 $E = \phi$
- 自环 (v_i, v_i)
- 重边



- 一个不含自环和重边的图称为简单图，以后如果没有特别声明主要讨论简单图。



图的定义—端的度数

- 对无向图的端 v_i ，与该端关联边的数目为该端的度数，记为： $d(v_i)$ 。
- 对有向图的端 v_i ， $d^+(v_i)$ 表示离开 v_i 的边数， $d^-(v_i)$ 表示进入 v_i 的边数。

• 性质5.1 $G = (V, E) \quad |V| = n \quad |E| = m$

– 对无向图 $\sum_{i=1}^n d(v_i) = 2m$

– 对有向图

$$\sum_{i=1}^n d^+(v_i) = \sum_{i=1}^n d^-(v_i) = m$$



图的定义—子图

- 给定图 $G = (V, E)$, 若 $V_1 \subseteq V, E_1 = \{(u, v) \in E \mid u, v \in V_1\}$ 称图 $G_1 = (V_1, E_1)$ 是 G 中由 V_1 生成的子图, 记为 $G[V_1]$ 。若 $E_2 \subseteq E_1$, 称 $G_2 = (V_1, E_2)$ 为 G 的子图。特别, 若子图的端点集合为 V , 这个图被称为图 G 的支撑子图。
- 若 $E_1 \subseteq E, V_1 = \{v \in V \mid v \text{ 是 } E_1 \text{ 中某边的端点}\}$ 称图 $G_1 = (V_1, E_1)$ 是 G 中由 E_1 生成的子图, 记为 $G[E_1]$ 。



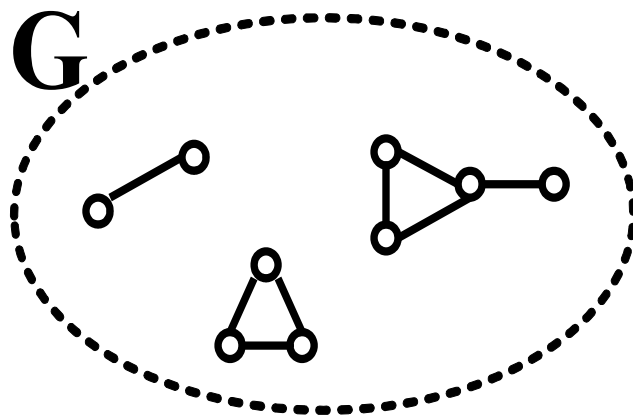
图的定义—图的连通性

- 考虑边的一个序列，相邻两边有公共端，如 (v_1, v_2) , (v_2, v_3) , (v_3, v_4) , $\dots\dots (v_i, v_{i+1})$ ，这个边序列称为**链**，链简单说就是一个连续轨迹。
- 没有重复边的链称为**简单链**；没有重复端的链称为**初等链或道路**；
- 若链的起点与终点重合，称之为**圈**；若道路的起点与终点重合，称之为**初等圈**。一般重点讨论**道路和初等圈**。



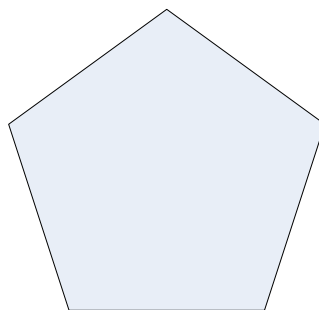
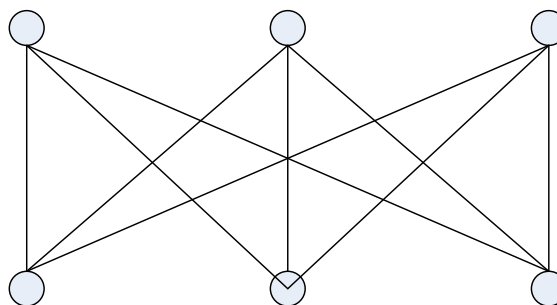
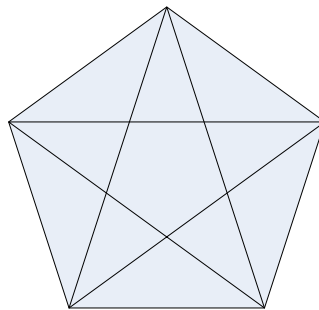
图的定义—连通图

- 任何两端间至少存在一条链的图，为连通图。否则，就是非连通图。
- 如：非连通图 G 有三个连通分支



图的例子

- 完全图 K_n
- 两部图
- 欧拉图
- 正则图





5.1.2 树

- 定义5.2 无圈的连通图称为树。
- 性质5.2 除单点树，至少有两个度数为1的端(悬挂点)。
- 性质5.3 任意树的边数 m 和端数 n 满足

$$m = n - 1$$



5.1.2 树

- 定理5.1 给定一个图 T ，若 $|V| = n$ ，
 $|E| = m$ ，则下面论断等价：
 - (1) T 是树；
 - (2) T 无圈，且 $m = n - 1$ ；
 - (3) T 连通，且 $m = n - 1$ 。



5.1.2 树

- 性质5.4 若 T 是树，则：
 - (1) T 是连通图，去掉任何一条边，图便分成两个且仅仅两个连通分支；
 - (2) T 是无圈图，但添加任何一条边，图便会包含一个且仅仅一个圈。
- 性质5.5 设 T 是树，则任何两点之间恰好有一条道路；反之，如图 T 中任何两点之间恰好有一条道路，则 T 为树。



5.1.2 树

- 如果树 T 是连通图 G 的子图, $T \subset G$, 且 T 包含 G 的所有端, 称 T 是 G 的支撑树或主树。
- 连通图一定有支撑树。
- 树边, 连枝。



5.1.3 割集

- 割集指的是某些端集或边子集，对连通图，去掉此类子集，图变为不连通。
- 定义5.3 割端与割端集
- 设 v 是图 G 的一个端，去掉 v 和其关联边后， G 的部分数增加，则称 v 是图 G 的割端。去掉一个端集合后， G 的部分数增加，这个端的集合称为割端集。



点连通度

- 对于连通图, 在众多的割端集中至少存在一个端数最少的割端集, 称为最小割端集。
- 最小割端集的端数目, 称为图的点连通度或连通度, 连通度用 α 表示。



线连通度

- 定义5.4 割边与割边集
- 设 e 是图 G 的一条边，去掉 e 后， G 的部分数增加，则称 e 是图 G 的割边。去掉一个边集合后， G 的部分数增加，这个边的集合称为割边集。
- 割边集中边数最少的割边集，称为最小割边集。最小割边集的边数目，称为线连通度，线连通度用 β 表示。



性质5.6

- 性质5.6 对于任意一个连通图 $G=(V,E)$, 若 $|V|=n$, $|E|=m$, δ 为最小度, 则

$$\alpha \leq \beta \leq \delta \leq \frac{2m}{n}$$



基本割集和基本圈

- 对于任何一个连通图 G ，设 T 为 G 的一个支撑树，每一条连枝决定的圈是基本圈。
- 确定了连通图的一个支撑树后，每条树边可以决定一个基本割集。
- 对于支撑树，去掉树上任何一条边，树便分为两个连通分支，从而将原图的端分为两个集合，这两个集合之间的所有边形成一个极小边割集，这个边割集称为基本割集。

基本割集和基本圈

- 基本圈和基本割集

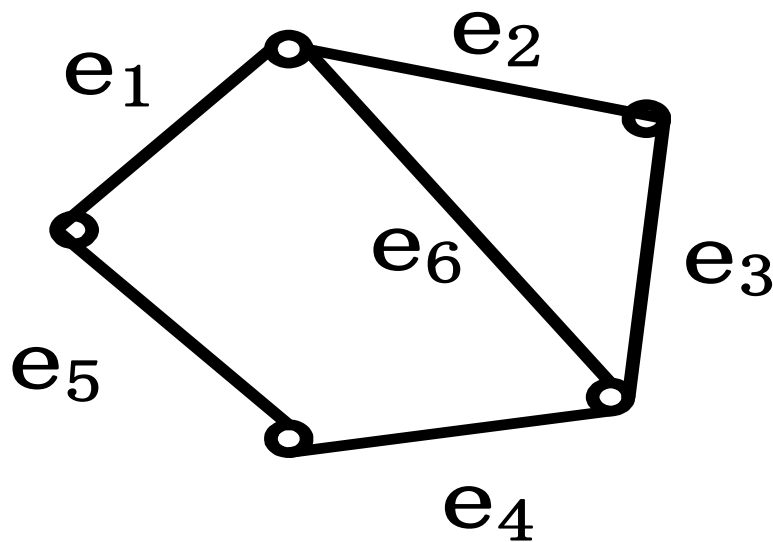


图5.3 基本圈和基本割集



基本割集和基本圈

- 基本圈和基本割集有许多应用, 首先通过集合的对称差运算, 由基本割集可以生成新的割集或它们的并集, 事实上可以证明生成所有的割集; 基本圈也有类似的性质。
- 例5.2 通过基本圈和基本割集分析求解电网络。



基本割集和基本圈

- 分析：在每个等电势的区域抽象一个端点，一个电网络为一个连通图。如果这个图有 n 个端点， m 条边，则有 $n-1$ 个独立电势变量。任取一个支撑树，有 $n-1$ 个基本割集。在每个基本割集上，根据基尔霍夫电流定律，流过基本割集电流的代数和为零。这 $n-1$ 个方程由于每个方程含有唯一的树边项，所以这 $n-1$ 个方程线性无关，通过这些方程可以求解所有的电势变量。
- 类似分析知道：有 $m-n+1$ 个回路电流方程



反圈

- 下面给出一个重要的概念：反圈。
- 定义5.5 反圈：给定图 $G=(V,E)$ ，若 $S,T \subseteq V$ ，记 $[S,T]_G = \{(u,v) \in E : u \in S, v \in T\}$ ；特别，当 $T = V \setminus S$ 时，将 $[S,T]_G$ 记为 $\Phi_G(S)$ 或 $\Phi(S)$ 。

设 X 是 V 的非空真子集，若 $\Phi_G(X) \neq \emptyset$ ，称 $\Phi_G(X)$ 为由 X 确定的反圈。

- 反圈是一种特殊的边割集



5.1.4 图的矩阵表示

- 下面将给出图的矩阵表示，主要介绍关联阵和邻接阵。

- 1 关联阵

设图G有 n 个端， m 条边，则全关联阵 $A_0 = [a_{i,j}]_{n \times m}$

其中，

$$\text{在有向图中} \begin{cases} a_{ij} = +1, e_j \text{与} v_i \text{关联, 离开} v_i \\ a_{ij} = -1, e_j \text{与} v_i \text{关联, 指向} v_i \\ a_{ij} = 0, e_j \text{与} v_i \text{不关联} \end{cases}$$

$$\text{在无向图中} \begin{cases} a_{ij} = 1, \text{若} e_j \text{与} v_i \text{关联} \\ a_{ij} = 0, \text{若} e_j \text{与} v_i \text{不关联} \end{cases}$$



关联阵

- 例5.3 考虑下面图5.4所表示的图

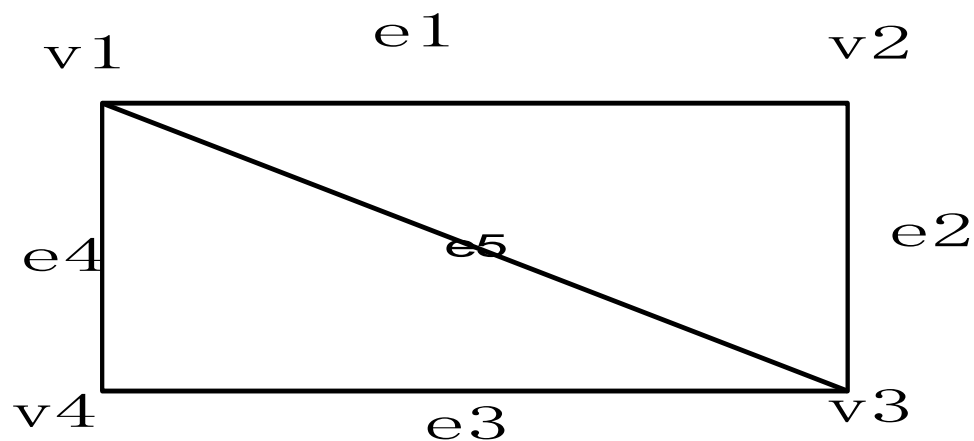


图5.4 例5.3图

$$A_0 = \begin{matrix} & \begin{matrix} e_1 & e_2 & e_3 & e_4 & e_5 \end{matrix} \\ \begin{matrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{matrix} & \begin{bmatrix} 1 & 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \end{bmatrix} \end{matrix}$$



关联阵

- 关联阵中每行对应一个端，每列对应一个边，由于完全表示了图中端集和边集的信息，所以关联阵是图的一个等价表示。
- 每行非零元个数等于相应端的度数，每列有两个1；
- 任意两行或两列互换得到的关联阵本质上是一个图。
- 将 A_0 中每列的任一个1改为-1，因为 n 行之和为零，所以最多只有 $n-1$ 行线性无关，再去掉任一行，得到关联阵 A ，这是一个 $(n-1) \times m$ 矩阵。

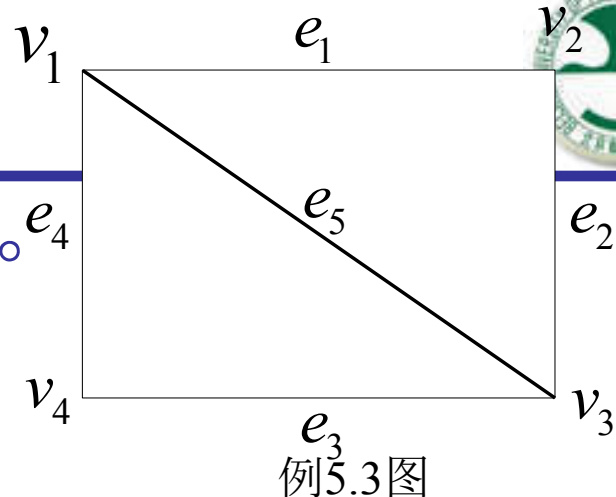


定理： 矩阵-树定理

- 可以证明 A 的每一个 $n-1$ 阶非奇异方阵一一对应一个支撑树（或称主树），并且该方阵的行列式的绝对值为1
- 定理5.2（矩阵-树定理）
用 A^T 表示 A 的转置, 无向图 G 的主树数目为
$$t(G) = \det(AA^T)$$
- $n-1$ 阶方阵 AA^T 可以直接写出, 主对角线的元素为相应端点的度数, 其余位置根据相应的端点之间是否有边取值为-1（或1）或0。



例：由关联阵A求右图
(例5.3 图)的支撑树数目。



- 继续例5.3，则

$$A_0 = \begin{bmatrix} 1 & 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \end{bmatrix} \xrightarrow{\text{去掉第一行 得到关联阵A}} A = \begin{matrix} v_2 \\ v_3 \\ v_4 \end{matrix} \begin{bmatrix} 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \end{bmatrix}$$

- 则支撑树的数目为：

$$t(G) = \det(AA^T) = \begin{matrix} v_2 & v_3 & v_4 \end{matrix} \begin{bmatrix} 2 & -1 & 0 \\ -1 & 3 & -1 \\ 0 & -1 & 2 \end{bmatrix} = 8$$



关联阵

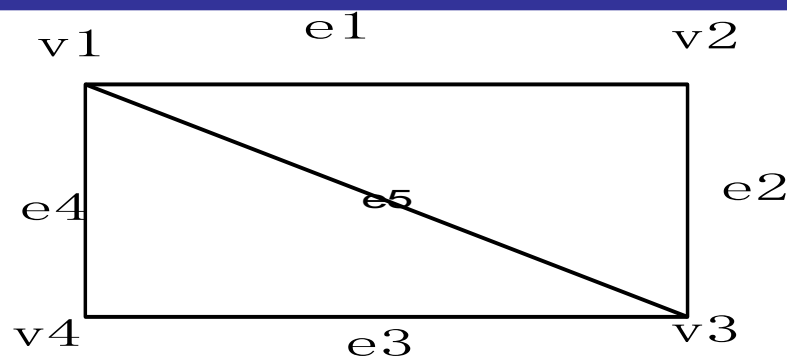
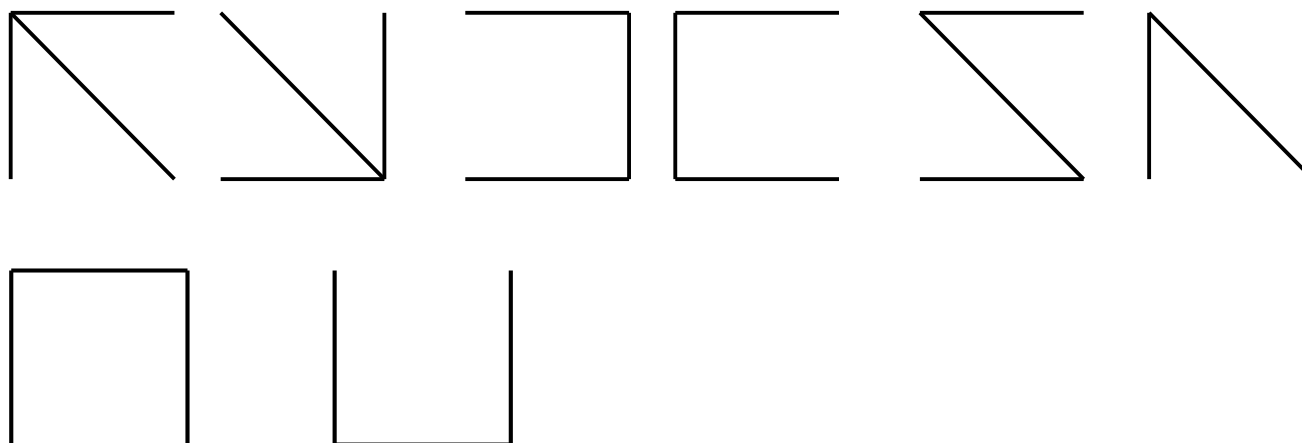


图5.4 例5.3图

- 共有8种支撑树如下：





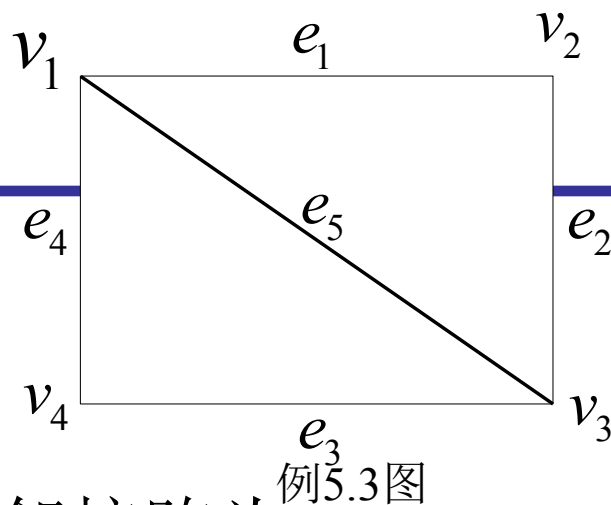
邻接阵

- 2 邻接阵
- 邻接阵是表示图的端与端关系的矩阵，其行和列都与端相对应。

令 $G = (V, E)$ 为 n 端， m 边的有向图，其邻接阵： $C = [c_{ij}]_{n \times n}$

$$\text{其中, } c_{ij} = \begin{cases} 1, & \text{若 } v_i \text{ 到 } v_j \text{ 有边} \\ 0, & \text{若 } v_i \text{ 到 } v_j \text{ 无边} \end{cases}$$

邻接阵举例



例5.3图

- 对于例5.3中的无向图，邻接阵为：

$$C = \begin{matrix} & \begin{matrix} v_1 & v_2 & v_3 & v_4 \end{matrix} \\ \begin{matrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{matrix} & \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix} \end{matrix}$$

- 对于无向图 $c_{ij} = c_{ji}$ 因此邻接阵为对称阵。

邻接阵C的特点

$$C = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}$$



- 从这个图可以看出：
 - (1) 矩阵的阶数就是图的端数，矩阵中1的个数就是图的有向边的个数；
 - (2) 对角线一般为0，因为是简单图，无自环；
 - (3) 对于无向图 $c_{ij} = c_{ji}$ ，因此邻接阵为对称阵。
- 邻接阵也包含了图的所有信息（端集、边集、每条边在哪两个端之间），是图的等价表示。

邻接阵的应用——判断图的连通性



- 如：考查 C^3 , 若 $C_{i,j}^3 = \sum_{k=1}^n \sum_{l=1}^n c_{i,k} c_{k,l} c_{l,j} \neq 0$ ，这是包含 n^2 项，有一项为1，必须满足 $c_{i,k} = c_{k,l} = c_{l,j} = 1$ 。即 i 到 j 之间存在链 $i \rightarrow k \rightarrow l \rightarrow j$
- 通过 C 可知各端点间是否可直达；通过 C^2 可知各端点间是否有长度为2的链；通过 C^3 可知各端点间是否有长度为3的链
- 通过计算 C 的各阶幂次可以得知图是否连通。



Warshall算法

- 矩阵P为判断矩阵，1表示连通，0为不连通
- (1)置新矩阵 $P := C$;
- (2)置 $i = 1$;
- (3)对所有的 j , 如果 $p(j, i) = 1$, 则对
 $k = 1, 2, \dots, n$, 有 $p(j, k) := p(j, k) \vee p(i, k)$;
- (4) $i = i + 1$;
- (5) 如 $n \geq i$ 转向步骤(3), 否则停止。



Warshall算法说明

- 置新矩阵 $P:=C$ ，说明该算法是基于邻接阵 C 来做的；
- i, j, k 都是 $1:n$ ，循环 n 次；
- 对所有的 j ，如果 $p(i, j)=1$ ，则说明 i 与 j 连通；只有当 $p(j, k)=0$ 且 $p(i, k)=1$ （端 i 和端 k 邻接）时才会进行更新，即 j 与 k 间可通过 i 转接连通。

$$k = 1, 2, \dots, n, p(j, k) := p(j, k) \cup p(i, k)$$

- P 最后的结果是：如果 $P(i, k)=1$ ，则说明端 i 和端 k 间有链。



Warshall算法实现举例

初始化矩阵 p 为邻接阵 C

for $k=1:n$ // k 为目的节点

for $j=1:n$ // j 为源节点

for $i=1:n$ // i 为转接节点

if $p(j,i)=1$

$\{ p(j,k) := p(j,k) \cup p(i,k)$

$i=i+1;$

$\}$

.....

小结



- 图的矩阵表示方法：关联阵，邻接阵
- 关联阵的应用
- 邻接阵的应用
- Warshall算法



主要内容

- 5.1 图论基础
 - 5.1.1 图的定义和基本概念
 - 5.1.2 树
 - 5.1.3 割集
 - 5.1.4 图的矩阵表示
- 5.2 最短路径问题
 - **5.2.1 最小支撑树（反圈法、避圈法、破圈法）**
 - 5.2.2 端间最短距离和路由（Dijkstra算法，Floyd算法）

• 5.3 网络流问题

• 5.4 双权问题



5.2 最短路径问题

- 上节中介绍的图只考虑了图顶点之间的关联性，本节将要对图的边和端赋予权值，讨论有权图。
- 权值在各种各样实际问题中有不同的物理意义，如费用，几何距离，容量等。
- 在本节中将讨论最小支撑树和最短路径问题等算法。



5.2.1 最小支撑树

- 给定连通图 $G = (V, E)$, $w(e)$ 是定义在 E 上的非负函数, $T = (V, E_T)$ 为 G 的一个支撑树。定义树 T 的权为 $w(T) = \sum_{e \in E_T} w(e)$ 。
- 最小支撑树问题就是求支撑树 T^* , 使 $w(T^*)$ 最小。下面介绍求最小支撑树的方法, 首先不加证明地引用定理5.3。



最小支撑树的特征

定理5.3 设 T^* 是 G 的支撑树,则如下论断等价:

- (1) T^* 是最小支撑树;
- (2)对 T^* 的任一树边 e , e 是由 e 所决定的基本割集或反圈中的最小权边;
- (3)对 T^* 的任一连枝 e , e 是由 e 所决定的基本圈中的最大权边。

- 这个定理描述了最小支撑树的特征。依照不同的逻辑, 可以有下面不同地具体做法
 - 反圈法和避圈法:(2)
 - 破圈法: (3)



Prim 算法—反圈法

- *Prim*(1957年)
- (1)任取一点作为初始的 $X^{(0)}$;
- (2)在反圈 $\Phi[X^{(k)}]$ 中选边的原则是:
 - 从 $\Phi[X^{(k)}]$ 中选一条权最小的边 (如果有
多条权最小的边, 则任选一条) ;
 - 将选出边的邻端并入 $X^{(k)}$ 形成 $X^{(k+1)}$;
- (3)若在某一步, $\Phi[X^{(k)}] = \phi$, 则 G 不含支
撑树; 若在某一步, $X^{(k)} = V$, 则由所
有被选边生成的树是最小支撑树。



Prim 算法理解

(2) 反圈是一种割边集，在反圈中选边，即在使得 $X(k)$ 成为非连通部分的割边集中选边。Prim算法从一个端点开始，通过在该端点的反圈中添加权最小的边，逐步使树长大；直至包含连通图的所有端，形成最小支撑树。

(3) 端遍历未完而反圈为空集，说明 G 非连通，则不含支撑树。



Prim算法举例

- 例5.4 求下图中的最小支撑树。

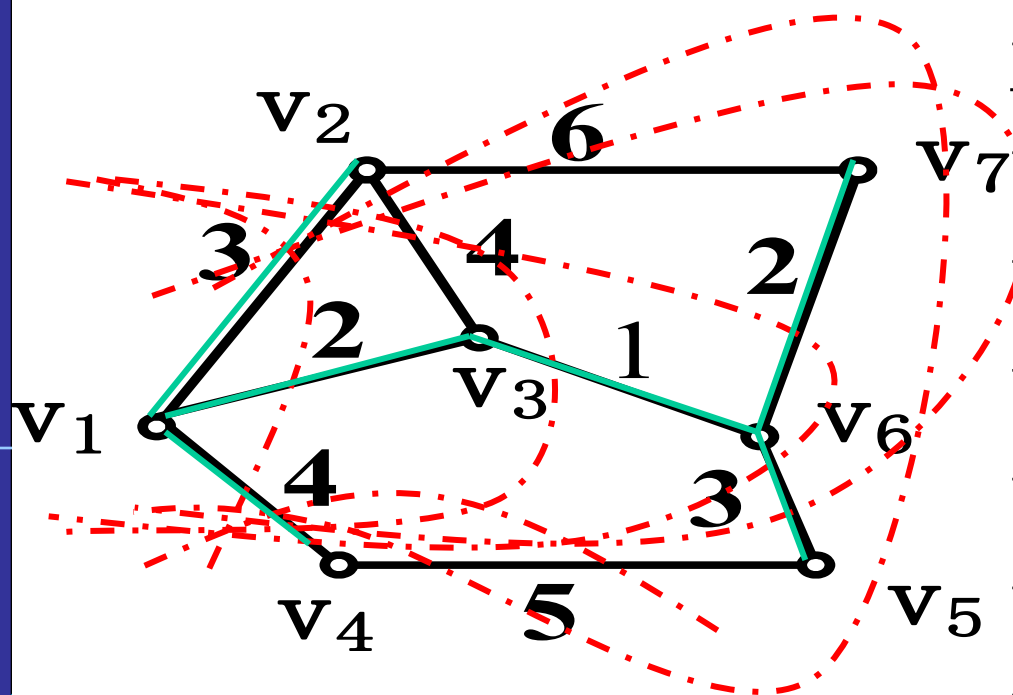


图5.5 例5.4

$\Phi(X^{(k)})$	
$X(k)$	边权
V1	3,2,4
+V3	3,4,1,4
+V6	3,4,2,3,4
+V7	3,4,6,3,4
+V5	3,4,6,5,4
+V2	4,5
+V4	

最小支撑树 $w(T^*)=15$



Prim算法的复杂度

- 算法一共进行 $n-1$ 步，每步使 k 个端与 $n-k$ 个端比较，最多需要 $k(n-k)-1$ 次比较然后可以选出反圈中权最小的边。所以计算量最多为：

$$\sum_{k=1}^{n-1} [k(n-k)-1] = \frac{1}{6}(n-1)(n-2)(n-3) = O(n^3)$$

- Prim算法是一个多项式(Polynomial)复杂度的算法
(非多项式复杂度—NP-hard问题)

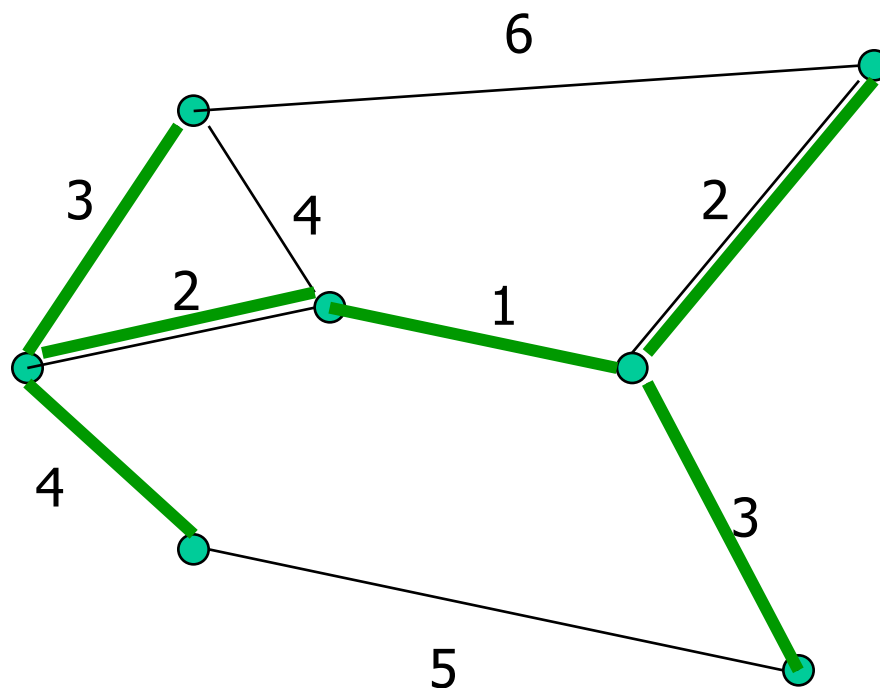


Kruskal算法—避圈法

- 设 $G^{(k)}$ 是 G 的无圈支撑子图, 开始 $G^{(0)} = (V, \Phi)$ 。
 若 $G^{(k)}$ 是连通的, 则它是最小支撑树; 若 $G^{(k)}$ 不连通, 取 $e^{(k)}$ 为这样的一边, 它的两个端点分属 $G^{(k)}$ 的两个不同连通分支, 并且权最小。令 $G^{(k+1)} = G^{(k)} + e^{(k)}$, 重复上述过程。
- 将所有边排序, 然后由小到大选边, 保持所选的边不生成圈, 如果选了 $n-1$ 条边, 则生成了一个最小支撑树。Kruskal算法的复杂度由对边的排序算法的复杂度决定。



避圈法举例



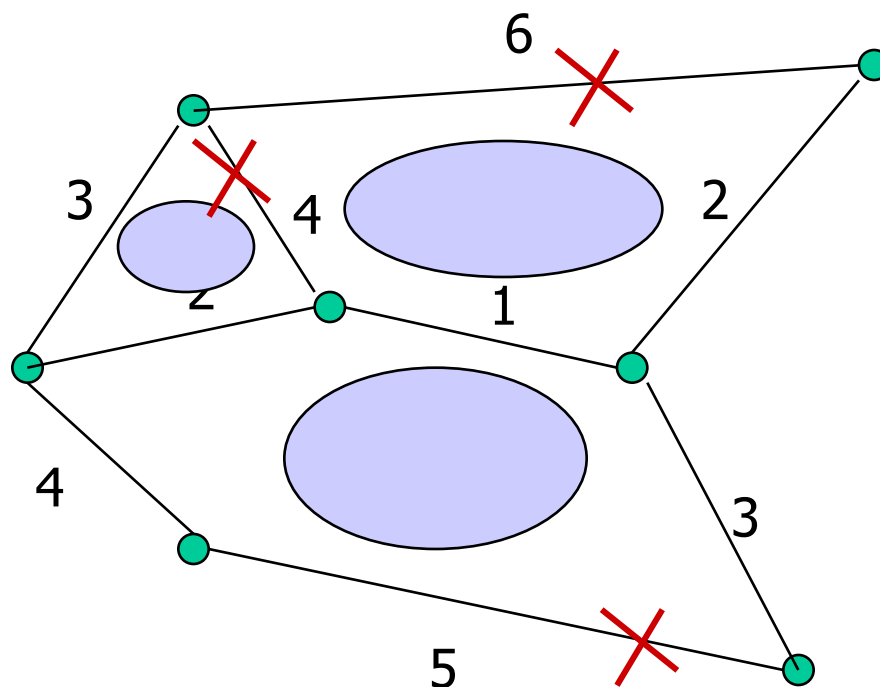


破圈法

- 设 $G^{(k)}$ 是 G 的连通支撑子图，开始 $G^{(0)} = G$ 。若 $G^{(k)}$ 中不含圈，则它是最小支撑树；若 $G^{(k)}$ 中包含圈，设 μ 是 $G^{(k)}$ 中的一个圈，取 μ 上的一条权最大的边 $e^{(k)}$ ，令 $G^{(k+1)} = G^{(k)} - e^{(k)}$ ，重复上述过程。
- 从连通图中先寻找圈，然后在圈中删去权最大的边，最后剩下的无圈连通图为最小支撑树



破圈法举例



破圈法



- 例5.5 对于一个无向图, 如何寻找其中的圈?
- 解:
 - 首先, 度为1的端点肯定不在任何圈上, 删去此类悬挂点。
 - 其次, 对于不含度为1的端的图, 从任意一个端出发漫游, 由于有限性, 端一定会重复, 而这就找到了一个圈。



主要内容

- 5.1 图论基础
 - 5.1.1 图的定义和基本概念
 - 5.1.2 树
 - 5.1.3 割集
 - 5.1.4 图的矩阵表示
- 5.2 最短路径问题
 - 5.2.1 最小支撑树（反圈法、避圈法、破圈法）
 - 5.2.2 端间最短距离和路由（*Dijkstra*算法, *Floyd*算法）

- 5.3 网络流问题
- 5.4 双权问题

5.2.2 端间最短路径和路由



- 已知图 $G = (V, E)$ ，每条边 (i, j) 有权 $w_{i,j} \geq 0$ 需要求网络中端点之间的最短距离和路由。这类问题分两种情况：
 - 1 寻找指定端至其它端的最短路径和路由，这个问题可以使用 *Dijkstra* 算法解决；
 - 2 寻找任意二端最短路径和路由，这个问题可以使用 *Floyd* 算法解决。



Dijkstra算法

- 图 $G = (V, E)$ 的每一边上有一个权 $w(e) \geq 0$
- 设 μ 是 G 中的一条链，定义链 μ 的权为： $w(\mu) = \sum_{e \in \mu} w(e)$ 。
- Dijkstra(1959年)算法可以简述如下：
- (1) 初始 $X^{(0)} = \{v_1\}$ ，记 $\lambda_1 = 0$ ，并且 v_1 的标号为 $(\lambda_1, 1)$ 。各端的标号记录了该端与源节点之间的距离以及到源节点之间的路由
- (2) 对任一边 $(i, l) \in \text{反圈 } \Phi[X^{(k)}] (v_i \in X^{(k)}, v_l \notin X^{(k)})$ 计算 $\lambda_i + w_{il}$ 的值。



Dijkstra算法

- 在 $\Phi[X^{(k)}]$ 中选一边，设为

$$(i_0, l_0)(v_{i_0} \in X^{(k)}, v_{l_0} \notin X^{(k)})$$

- 使 $\lambda_{i_0} + w_{i_0 l_0} = \min_{(i,l) \in \Phi[X^{(k)}]} (\lambda_i + w_{il})$ ，并令 $\lambda_{l_0} = \lambda_{i_0} + w_{i_0 l_0}$
并且 v_{l_0} 的标号为 (λ_{l_0}, i_0) 。

各端的标号记录了该端与源节点之间的距离以及到源节点之间的路由

(3)当出现下面情况之一时停止。

情况1：目的端 v_j 满足 $v_j \in X^{(k)}$ ；

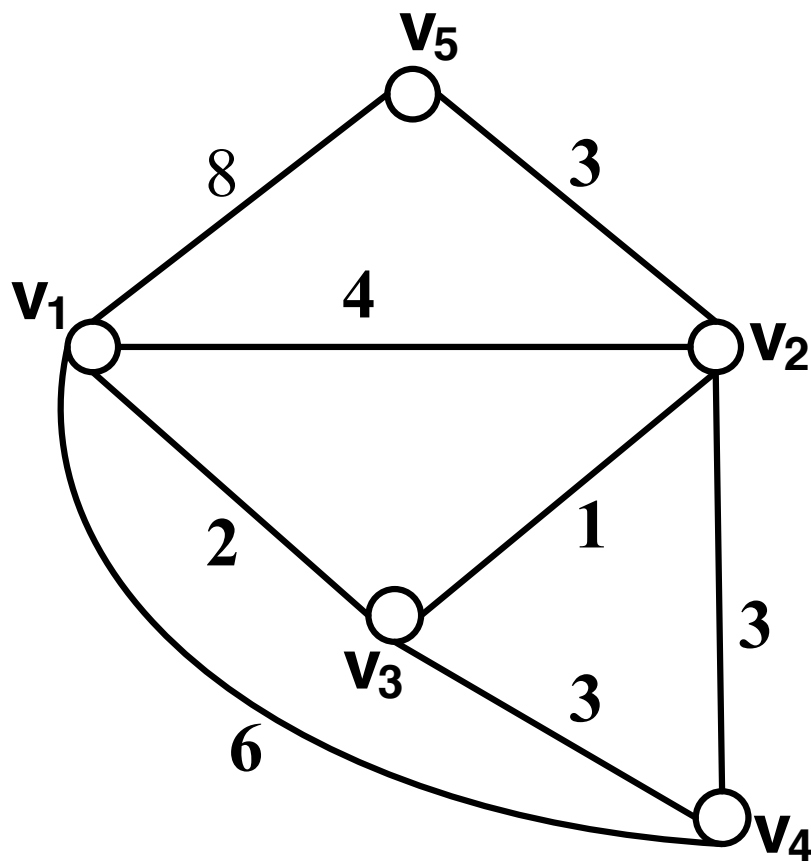
情况2：目的端 v_j 满足 $v_j \notin X^{(k)}$ ，但

$$\Phi[X^{(k)}] = \phi。$$



Dijkstra算法举例

- 例5.6 在下图中求 v_1 到其余端点的最短距离和路由。





D算法举例

- 例5.6 在下图中求 v_1 到其余端点的最短距离和路由。

$$\lambda_i + w_{i,l}$$

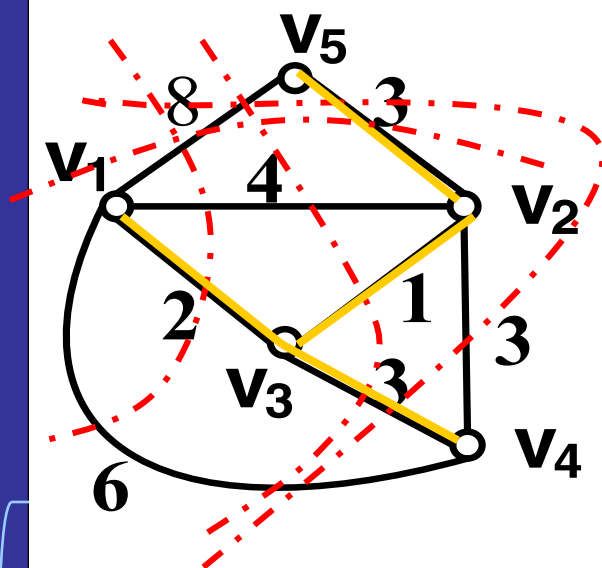


图5.6 例5.5

v_1	v_5	v_2	v_3	v_4	置定端 l_0	距离 λ_{l_0}	路由 i_0
0					V1		
	8	4	2	6	V3	2	V1
	8	3		5	V2	3	V3
	6			5	V4	5	V3
	6				V5	6	V2

(1) 标号置定法 (Label-setting)

(2) 回溯路由

*Dijkstra*算法



- 对于*Dijkstra*算法, 提出若干问题如下:
- (1) 如果端点有权如何处理?
- (2) 如果边的权可正可负, 算法是否仍然有效?
- (3) 算法是否对有向图也适用?



*Dijkstra*算法

- 例5.7 深度优先和广度优先搜索。
- 如果要搜索的环境为图 $G=(V,E)$ ，每条边赋权1，搜索的起点为 v_1 ，应用*Dijkstra*算法求 v_1 到任意端 v_i 的距离，距离 λ_i 表示了端 v_i 的“代”数。
- 广度优先搜索可以简述如下：
(1) 开始取 v_1 作为 $X^{(0)}$ 。



广度优先搜索

- (2) 在 $\Phi[X^{(k)}]$ 中选边时，遵守如下原则：首先在 $X^{(k)}$ 中选一个“代”数最小的端，如 v_i ，然后选所有以 v_i 为端点的边。如果 v_i 的标号为 (λ_i, l) ，则选中边的相邻端的标号应为 $(\lambda_i + 1, i)$ ；将新的端点加入到 $X^{(k)}$ 形成 $X^{(k+1)}$ 。
- (3) 如果在某一步， $\Phi[X^{(k)}] = \emptyset$ ，表明图不连通；如果在某一步， $X^{(k)} = V$ ，结束。
- 注意，标号记录了该端的“代”数和搜索到该端的路由。



深度优先搜索

- (1) 开始取 v_1 作为 $X^{(0)}$ 。
- (2) 在 $\Phi[X^{(k)}]$ 中选边时，遵守如下原则：
只选一条边，该边在 $X^{(k)}$ 中的端的“代”数最大；将新的端点加入到 $X^{(k)}$ 形成 $X^{(k+1)}$ 。
- (3) 如果在某一步， $\Phi[X^{(k)}] = \emptyset$ ，表明图不连通；如果在某一步， $X^{(k)} = V$ ，结束。

Floyd算法



- 定理5.4 对于图 G , 如果 $w(i, j)$ 表示端 i 和端 j 之间的可实现的距离, 那么 $w(i, j)$ 表示端 i 和 j 之间的最短距离当且仅当对于任意 i, k, j , 有

$$w(i, j) \leq w(i, k) + w(k, j)$$

证明: 首先, 如果 $w(i, j)$ 表示端 i 和 j 之间的最短距离, 则

$$w(i, j) \leq w(i, k) + w(k, j)$$



Floyd算法

- 下面考虑充分性:
- 若 μ 是任一个从端 i 到端 j 的链,
 $\mu = (i, i_1, i_2, \dots, i_k, j)$, 则反复应用充分条件,
有

$$\begin{aligned} w(i, j) &\leq w(i, i_1) + w(i_1, j) \\ &\leq w(i, i_1) + w(i_1, i_2) + w(i_2, j) \\ &\leq \dots \\ &\leq w(i, i_1) + w(i_1, i_2) + \dots + w(i_k, j) \\ &= w(\mu) \end{aligned}$$



Floyd算法

- 因为 $w(i, j)$ 表示端 i 和 j 之间的可实现的距离，则 $w(i, j)$ 表示端 i 和 j 之间的最短距离。
- F算法通过迭代运算消除不满足定理5.4的情况。



Floyd算法

- 给定图 G 及其边 (i, j) 的权 $w_{i,j}$ ($1 \leq i \leq n, 1 \leq j \leq n$)
- F_0 : 初始化距离矩阵 $W^{(0)}$ 和路由矩阵 $R^{(0)}$
- 其中:

$$w_{ij}^{(0)} = \begin{cases} w_{ij} & \text{若 } e_{ij} \in E \\ \infty & \text{若 } e_{ij} \notin E \\ 0 & \text{若 } i = j \end{cases}$$

$$r_{ij}^{(0)} = \begin{cases} j & \text{若 } w_{ij}^{(0)} \neq \infty \\ 0, & \text{其它} \end{cases}$$

(1) W 为 $n \times n$ 端端方阵

(2) $W(0), R(0)$ ——只取直通过路由, 未考虑转接



Floyd算法

- F1: 已求得 $W^{(k-1)}$ 和 $R^{(k-1)}$, 依据下面的迭代求 $W^{(k)}$ 和 $R^{(k)}$:

$$w_{i,j}^{(k)} = \min(w_{i,j}^{(k-1)}, w_{i,k}^{(k-1)} + w_{k,j}^{(k-1)})$$

$$r_{i,j}^{(k)} = \begin{cases} r_{i,k}^{(k-1)} & \text{若 } w_{i,j}^{(k)} < w_{i,j}^{(k-1)} \\ r_{i,j}^{(k-1)} & \text{若 } w_{i,j}^{(k)} = w_{i,j}^{(k-1)} \end{cases}$$

- ✓ 元素计算由 $W^{(k-1)} \rightarrow W^{(k)}$, 以 V_k 为转接端, 无链的可有链, 原长链可能缩短



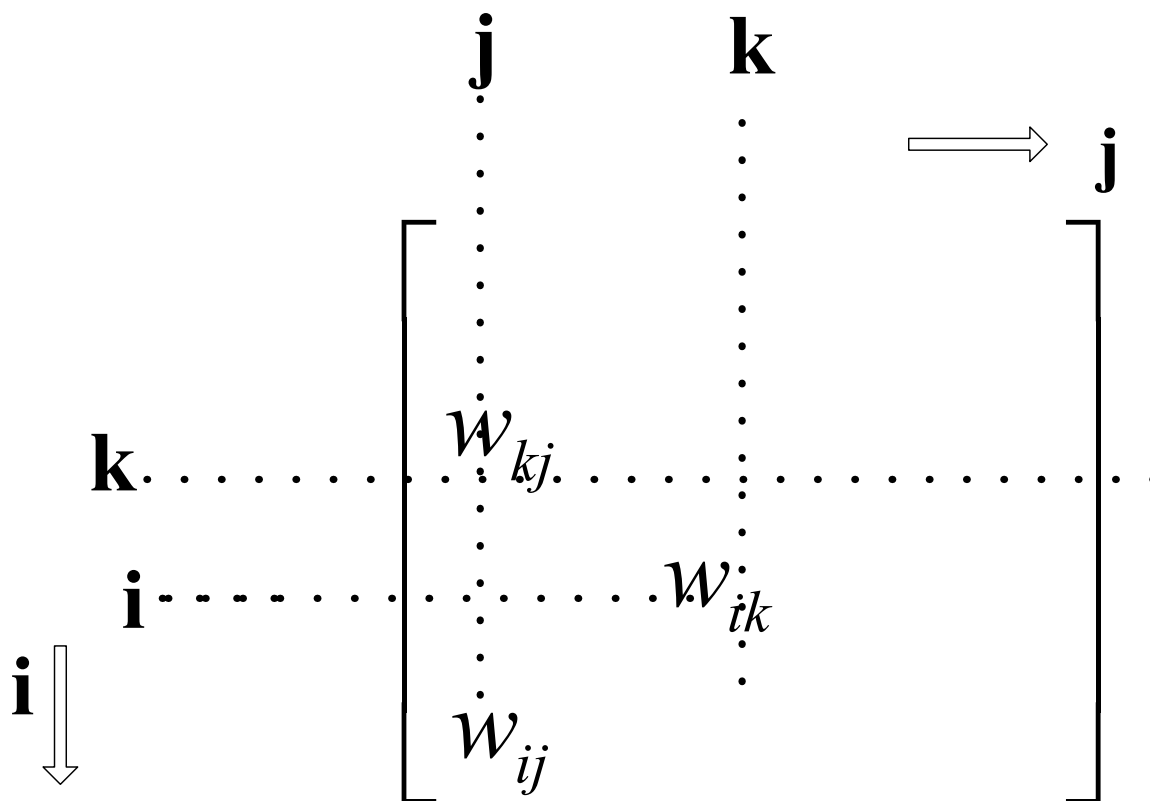
- F2: 若 $k < n$, 重复F1 ; $k = n$, 则终止。

✓ 依次取 $V_1, V_2, \dots, V_n$ 做转接, 依次得: $W^{(1)}, W^{(2)}, \dots, W^{(n)}$



F算法的W矩阵更新方法

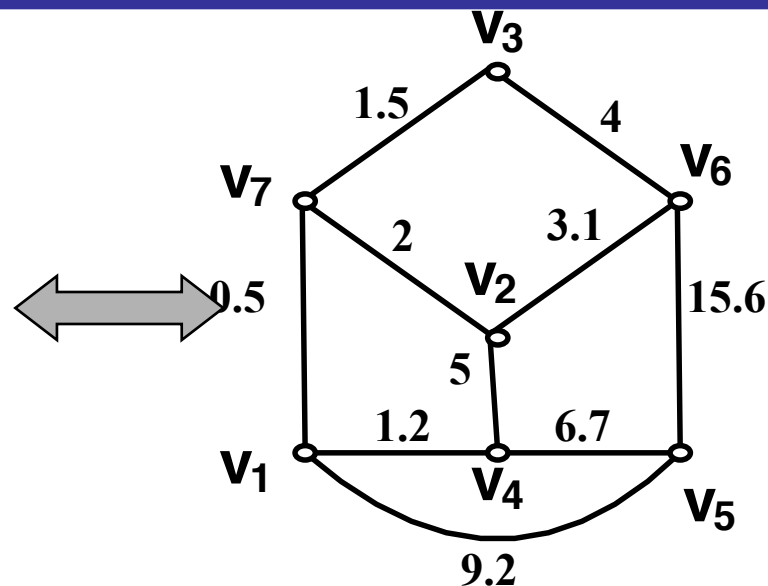
$$w_{i,j}^{(k)} = \min(w_{i,j}^{(k-1)}, w_{i,k}^{(k-1)} + w_{k,j}^{(k-1)})$$





例5.8 图G的距离矩阵如下，用Floyd算法求任意端间最短距离和路由，并求中心和中点。

$$w^{(0)} = \begin{bmatrix} 0 & \infty & \infty & 1.2 & 9.2 & \infty & 0.5 \\ \infty & 0 & \infty & 5 & \infty & 3.1 & 2 \\ \infty & \infty & 0 & \infty & \infty & 4 & 1.5 \\ 1.2 & 5 & \infty & 0 & 6.7 & \infty & \infty \\ 9.2 & \infty & \infty & 6.7 & 0 & 15.6 & \infty \\ \infty & 3.1 & 4 & \infty & 15.6 & 0 & \infty \\ 0.5 & 2 & 1.5 & \infty & \infty & \infty & 0 \end{bmatrix}$$



$$r_{ij}^{(0)} = \begin{cases} j & \text{若 } w_{ij}^{(0)} \neq \infty \\ 0, & \text{其它} \end{cases} \Rightarrow R^{(0)} =$$

$$\begin{bmatrix} 0 & 0 & 0 & 4 & 5 & 0 & 7 \\ 0 & 0 & 0 & 4 & 0 & 6 & 7 \\ 0 & 0 & 0 & 0 & 0 & 6 & 7 \\ 1 & 2 & 0 & 0 & 5 & 0 & 7 \\ 1 & 0 & 0 & 4 & 0 & 6 & 7 \\ 0 & 2 & 3 & 0 & 5 & 0 & 0 \\ 1 & 2 & 3 & 0 & 0 & 0 & 0 \end{bmatrix}$$



- 解：距离矩阵包含了邻接矩阵和权的所有信息，依照Floyd算法计算结果如下：

$$w^{(0)} = \begin{bmatrix} 0 & \infty & \infty & 1.2 & 9.2 & \infty & 0.5 \\ \infty & 0 & \infty & 5 & \infty & 3.1 & 2 \\ \infty & \infty & 0 & \infty & \infty & 4 & 1.5 \\ 1.2 & 5 & \infty & 0 & 6.7 & \infty & \infty \\ 9.2 & \infty & \infty & 6.7 & 0 & 15.6 & \infty \\ \infty & 3.1 & 4 & \infty & 15.6 & 0 & \infty \\ 0.5 & 2 & 1.5 & \infty & \infty & \infty & 0 \end{bmatrix} \quad k=1$$

$$w_{i,j}^{(k)} = \min(w_{i,j}^{(k-1)}, w_{i,k}^{(k-1)} + w_{k,j}^{(k-1)})$$

$$w^{(1)} = \begin{bmatrix} 0.00 & 100 & 100 & 1.20 & 9.20 & 100 & 0.50 \\ 100 & 0.00 & 100 & 5.00 & 100 & 3.10 & 2.00 \\ 100 & 100 & 0.00 & 100 & 100 & 4.00 & 1.50 \\ 1.20 & 5.00 & 100 & 0.00 & 6.70 & 100 & \underline{1.70} \\ 9.20 & 100 & 100 & 6.70 & 0.00 & 15.60 & \underline{9.70} \\ 100 & 3.10 & 4.00 & 100 & 15.60 & 0.00 & 100 \\ 0.50 & 2.00 & 1.50 & \underline{1.70} & \underline{9.70} & 100 & 0.00 \end{bmatrix}$$



R矩阵更新

$$R^{(0)} = \begin{bmatrix} 0 & 0 & 0 & 4 & 5 & 0 & 7 \\ 0 & 0 & 0 & 4 & 0 & 6 & 7 \\ 0 & 0 & 0 & 0 & 0 & 6 & 7 \\ 1 & 2 & 0 & 0 & 5 & 0 & 7 \\ 1 & 0 & 0 & 4 & 0 & 6 & 7 \\ 0 & 2 & 3 & 0 & 5 & 0 & 0 \\ 1 & 2 & 3 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$\Rightarrow R^{(1)} = \begin{bmatrix} 0 & 0 & 0 & 4 & 5 & 0 & 7 \\ 0 & 0 & 0 & 4 & 0 & 6 & 7 \\ 0 & 0 & 0 & 0 & 0 & 6 & 7 \\ 1 & 2 & 0 & 0 & 5 & 0 & \underline{1} \\ 1 & 0 & 0 & 4 & 0 & 6 & \underline{1} \\ 0 & 2 & 3 & 0 & 5 & 0 & 0 \\ 1 & 2 & 3 & \underline{1} & \underline{1} & 0 & 0 \end{bmatrix}$$

最后结果及验证

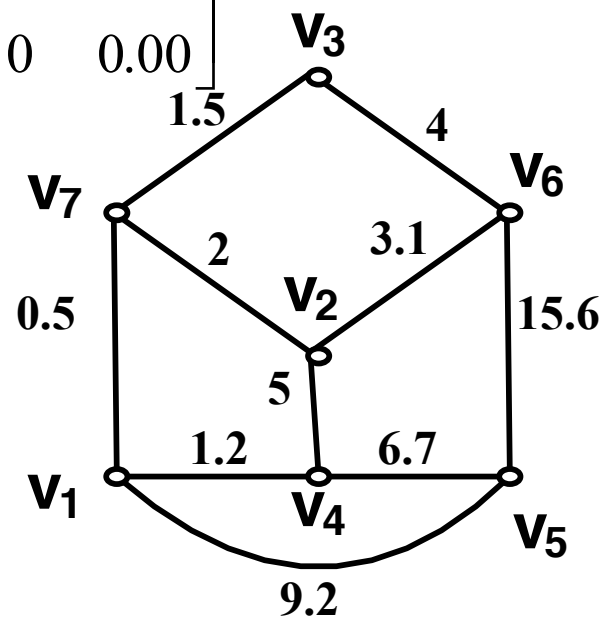


$$w^{(7)} = \begin{bmatrix} 0.00 & 2.50 & 2.00 & 1.20 & 7.90 & 5.60 & 0.50 \\ 2.50 & 0.00 & 3.50 & 3.70 & 10.40 & 3.10 & 2.00 \\ 2.00 & 3.50 & 0.00 & 3.20 & 9.90 & 4.00 & 1.50 \\ 1.20 & 3.70 & 3.20 & 0.00 & 6.70 & 6.80 & 1.70 \\ 7.90 & 10.40 & 9.90 & 6.70 & 0.00 & 13.50 & 8.40 \\ 5.60 & 3.10 & 4.00 & 6.80 & 13.50 & 0.00 & 5.10 \\ 0.50 & 2.00 & 1.50 & 1.70 & 8.40 & 5.10 & 0.00 \end{bmatrix}$$

⇒ 端间距离

$$R^{(7)} = \begin{bmatrix} 0 & 7 & 7 & 4 & 4 & 7 & 7 \\ 7 & 0 & 7 & 7 & 7 & 6 & 7 \\ 7 & 7 & 0 & 7 & 7 & 6 & 7 \\ 1 & 1 & 1 & 0 & 5 & 1 & 1 \\ 4 & 4 & 4 & 4 & 0 & 4 & 4 \\ 2 & 2 & 3 & 2 & 2 & 0 & 2 \\ 1 & 2 & 3 & 1 & 1 & 2 & 0 \end{bmatrix}$$

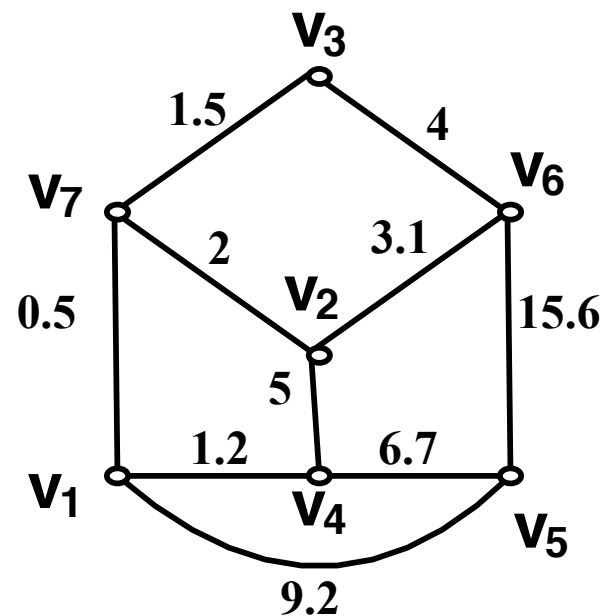
⇒ 端间路由





路由查找举例

$$R^{(7)} = \begin{bmatrix} 0 & 7 & 7 & 4 & 4 & 7 & 7 \\ 7 & 0 & 7 & 7 & 7 & 6 & 7 \\ 7 & 7 & 0 & 7 & 7 & 6 & 7 \\ 1 & 1 & 1 & 0 & 5 & 1 & 1 \\ 4 & 4 & 4 & 4 & 0 & 4 & 4 \\ 2 & 2 & 3 & 2 & 2 & 0 & 2 \\ 1 & 2 & 3 & 1 & 1 & 2 & 0 \end{bmatrix}$$



- 找v1→v4: 路由查得r14=4, 不转接v1→v4;
- 找v3→v4: 路由查得r34=7, 经7转接;
 - 查v3→v7得r37=7, 无转接。
 - 查v7→v4得r74=1, 经1转接: 查v7→v1得1, 无转接; 查v1→v4得4, 无转接。
 - 路由为v3→v7→v1→v4
- 找v4→v6:



图的中心与中点

- 已知图 $G=(V,E)$ 为权图, 根据 $Floyd$ 算法的结果可以定义网络的中心和中点。
- 中心
 - 对每个端点 i , 先求 $\max_j(w_{i,j}^{(n)})$
 - 此值最小的端称为网的中心, 即满足下式的端:

$$\max_j(w_{i^*,j}^{(n)}) = \min_i[\max_j(w_{i,j}^{(n)})]$$

网的中心宜作维修中心和服务中心。



- 中点
 - 对每个端点 i , 计算 $s_i = [\sum_j w_{i,j}^{(n)}]$, 然后求出
的 s_i 最小值, 相应的端点为中点。

网的中点宜作全网的交换中心。



上例中的中心和中点

- 对于中心和中点，根据的计算结果可以得到：

	$\max_j (w_{ij}^{(7)})$							$\sum_{j=1}^7 w_{ij}^{(7)}$	
$w^{(7)} =$	0.00	2.50	2.00	1.20	7.90	5.60	0.50	7.9	19.7
	2.50	0.00	3.50	3.70	10.40	3.10	2.00	10.4	25.2
	2.00	3.50	0.00	3.20	9.90	4.00	1.50	9.9	24.1
	1.20	3.70	3.20	0.00	6.70	6.80	1.70	<u>6.8</u>	23.2
	7.90	10.40	9.90	6.70	0.00	13.50	8.40	13.5	56.8
	5.60	3.10	4.00	6.80	13.50	0.00	5.10	13.5	38.1
	0.50	2.00	1.50	1.70	8.40	5.10	0.00	8.4	<u>19.2</u>
								中心	中点

- 从而，图的中心为 v_4 ，中点为 v_7

Floyd算法



- 对于Floyd算法, 同样提出若干问题如下:
 - (1) 如果端点有权如何处理?
 - (2) 如果边的权可正可负, 算法是否仍然有效?
 - (3) 算法是否对有向图也适用?
- 问题1和3在Dijkstra算法中有过讨论, 这里重点讨论问题(2)。



5.3 网络流量问题

- 网络的目的是把一定的业务流从源端送到宿端，流量分配的优劣将直接关系到网络的使用效率和相应的经济效益。
- 网络的流量分配受限于网络的拓扑结构，边和端的容量，流量分配和路由规划关系密切。
- 本节中关于流量问题的内容均在有向图上考虑，并且均是单商品流问题，即网络中需要安排的只有一种商品或业务；



5.3.1 基本概念

- 给定一个有向图 $G = (V, E)$, $c(e)$ 是定义在边集合 E 上一个非负函数, 称为容量; 边 $e_{i,j}$ 的容量 $c_{i,j}$ 表示这条边能通过的最大流量。
- 设 $f = \{f_{i,j}\}$ 是上述网络的一个流, 该流有一个源 v_s 和一个宿 v_t , 若能满足下述二个限制条件, 称为可行流。



可行流的条件

- (1) 非负有界性: 对任意边 $e_{i,j}$ 有,

$$0 \leq f_{i,j} \leq c_{i,j}$$

- (2) 连续性: 对任意端 v 有,

$$\sum_{(i,j) \in E} f_{ij} - \sum_{(j,i) \in E} f_{ji} = \begin{cases} F & v_i \text{ 为源端 } v_s \\ -F & v_i \text{ 为宿端 } v_t \\ 0 & \text{其他} \end{cases}$$



最大流问题

- $v(f) = F$ 为源宿间流的总流量。需要解决的基本问题分为两类:

1 最大流问题

在确定流的源和宿的情况下, 求一个可行流 f , 使 $v(f) = F$ 为最大。



最小费用流问题

- 2 最小费用流问题

如果边 $e_{i,j}$ 的单位流费用为 $d_{i,j}$, 流 f 的费用为: $c = \sum_{(i,j) \in E} d_{i,j} f_{i,j}$

最小费用流问题

在确定流的源和宿的情况下, 求一个可行流 f , 使 C 为最小。

割量



- 下面介绍割量和可增流路的概念。
- 设 X 是 V 的真子集，且 $v_s \in X$ $v_t \in X^c$ ， (X, X^c) 表示起点和终点分别在 X 和 X^c 的边集合，这是一个带方向的反圈或割集，割集的正方向为从 v_s 到 v_t 。割量 $C(X, X^c)$ 定义为这个割集中所有边容量的和：

$$C(X, X^c) = \sum_{v_i \in X, v_j \in X^c} C_{ij}$$



边的流量和

- 对可行流 $f = \{f_{i,j}\}$:

$f(X, X^c)$ 表示前向边的流量和,

$$f(X, X^c) = \sum_{v_i \in X, v_j \in X^c} f_{i,j} \quad ,$$

$f(X^c, X)$ 表示反向边的流量和,

$$f(X^c, X) = \sum_{v_i \in X^c, v_j \in X} f_{i,j} \quad ;$$



性质5.7

- 则源为 v_s 宿为 v_t 的任意流 f 有:
- 1. $v(f) = f(X, X^c) - f(X^c, X)$, 其中 $v_s \in X$,
 $v_t \in X^c$ 对任 $v_i \in X$:

$$\sum_{v_j \in V} f_{ij} - \sum_{v_j \in V} f_{ji} = \begin{cases} F & v_i = v_s \\ 0 & v_i \neq v_s \end{cases}$$

- 对所有 $v_i \in X$,将上述等式求和:

$$\begin{aligned} \sum_{v_i \in X} \sum_{v_j \in V} f_{ij} - \sum_{v_i \in X} \sum_{v_j \in V} f_{ji} &= \sum_{v_i \in X} \sum_{v_j \in X^c} f_{ij} - \sum_{v_i \in X} \sum_{v_j \in X^c} f_{ji} \\ &= f(X, X^c) - f(X^c, X) = F = v(f) \end{aligned}$$



性质5.7

- 2. $F \leq C(X, X^c)$

由 $f(X, X^c)$ 非负, 可得:

$$\begin{aligned} F &= f(X, X^c) - f(X^c, X) \\ &\leq f(X, X^c) \\ &\leq C(X, X^c) \end{aligned}$$



可增流路

- 下面讨论可增流路的概念。
- 从端s到端t的一个路，有一个自然的正方向，然后将路上的边分为两类：前向边集合和反向边集合。对于某条流，若在某条路中，前向边均不饱和 ($f_{i,j} < c_{i,j}$)，反向边均有非0流量 ($f_{i,j} \neq 0$)，称这条路为可增流路。
- 在可增流路上增流不影响连续性条件，也不改变其它边上的流量，同时可以使从源端到宿端的流量增大。



5.3.2 最大流问题

- 所谓最大流问题，在确定流的源端和宿端的情况下，求一个可行流 f ，使 $v(f)$ 为最大。对于一个网络，求最大流的方法采用可增流路的方法，下面的定理5.5为这种方法提供了保证。



5.3.2 最大流问题

- 定理5.5 (最大流-最小割定理)
- 可行流 $f^* = \{f_{ij}^*\}$ 为最大流当且仅当 G 中不存在从 v_s 到 v_t 的可增流路。
- 证明:
- 必要性: 设 f^* 为最大流, 如果 G 中存在关于 f^* 的从 v_s 到 v_t 的可增流路 μ 。

$$\theta = \min\{\min_{(i,j) \in \mu^+} (c_{i,j} - f_{i,j}^*), \min_{(i,j) \in \mu^-} (f_{i,j}^*)\} > 0$$

- 构造一个新流 f 如下:
- 如果 $(i,j) \in \mu^+$, $f_{i,j} = f_{i,j}^* + \theta$



5.3.2 最大流问题

如果 $(i, j) \in \mu^-$, $f_{i,j} = f_{i,j}^* - \theta$

如果 $(i, j) \notin \mu$, $f_{i,j} = f_{i,j}^*$

不难验证新流 f 为一个可行流, 而且
 $v(f) = v(f^*) + \theta$, 矛盾。

- 充分性: 设 f^* 为可行流, G 中不存在关于这个流的可增流路。



5.3.2 最大流问题

- 令 $X^* = \{v | G \text{ 中存在从 } v_s \text{ 到 } v \text{ 的可增流路}\}$, 从而 $v_s \in X^*, v_t \notin X^*$ 。
- 对于任意边 $(i, j) \in (X^*, X^{*c})$, 有 $f_{i,j}^* = c_{i,j}$
- 对于任意边 $(i, j) \in (X^{*c}, X^*)$, 有 $f_{i,j}^* = 0$
- 这样, $v(f^*) = c(X^*, X^{*c})$, 那么流 f^* 为最大流, (X^*, X^{*c}) 为最小割。证毕。
- 性质5.8: 如果所有边的容量为整数, 则必定存在整数最大流。



5.3.2 最大流问题

- 求最大流的基本思想是：
 在一个可行流的基础上， v_s 找 v_t 到的可增流路，然后在此路上增流，直至无可增流路时，停止。
- M算法：从任一可行流开始，通常以零流开始。
- （1）标志过程：从 v_s 开始给邻端加标志，加上标志的端称已标端；



5.3.2 最大流问题

- (2) 选查过程：从 v 开始选查已标未查端；查某端，即标其可能增流的邻端；所有邻端已标，则该端已查。标志宿端，则找出一条可增流路到宿端，进入增流过程。
- (3) 增流过程：在已找到的可增流路上增流。



5.3.2 最大流问题

- 步骤:
- M_0 : 初始令 $f_{i,j} = 0$;
- M_1 : 标源端 $v_s : (+, s, \infty)$;
- M_2 : 从 v_s 始, 查已标未查端 v_i , 即标 v_i
- 的满足下列条件的邻端 v_j ,
- 若 $(v_i, v_j) \in E$, 且 $c_{i,j} > f_{i,j}$, 标 v_j 为: $(+, i, \varepsilon_j)$
- 其中 $\varepsilon_j = \min\{c_{i,j} - f_{i,j}, \varepsilon_i\}$, ε_i 为 v_i 已标值。
- 若 $(v_j, v_i) \in E$, 且 $f_{j,i} > 0$, 标 v_j 为: $(-, i, \varepsilon_j)$
- 其中 $\varepsilon_j = \min\{f_{j,i}, \varepsilon_i\}$, 其余 v_j 端不标。



5.3.2 最大流问题

- 所有能加标的邻端 v_j 已标，则称 v_i 已查。
- 倘若所有端已查且宿端未标，则算法终止。
- M3: 若宿端 v_t 已标，则沿该可增流路增流。
- M4: 返回M1。



5.3.2 最大流问题

- 上面的算法是针对有向图且端无限制的情况。若是有无向边，端容量及多源多宿的情况，可以进行一些变换，化为上述标准情形。这个算法的复杂度不是多项式的，但是经过简单改进后算法为多项式复杂度。
- 需要解决下面的情况：
 - 1.端有容量限制；
 - 2.无向边；
 - 3.多源多宿；



5.3.3 最小费用流问题

- 如果网络为图 $G = (V, E)$ ，源端为 v_s ，宿端为 v_t ，边 (i, j) 的单位流费用为 $d_{i,j}$ ，流 f 的费用为：

$$C = \sum_{(i,j) \in E} d_{i,j} f_{i,j}$$

- 所谓最小费用流问题：
- 在确定流的源和宿的情况下，求一个可行流 f ，使 C 为最小。



5.3.3 最小费用流问题

- 最小费用流问题是线性规划问题，但也可用图论方法求解，效率更高。对于它的存在性可以这样理解，流量为 F 的可行流一般不是唯一的，这些不同的流的费用一般也不一样，有一个流的费用最小。
- 寻找最小费用流，可以用负价环法算法 (*Klein*, 1967年)。所谓负价环的意义如下：负价环为有向环，同时环上费用的和为负。负价环算法的具体步骤如下：



5.3.3 最小费用流问题

- K0: 在图 G 上找任意流量为 F 的可行流 f ;
- K1: 做流 f 的补图; 做补图的方法如下:
对于所有边 $e_{i,j}$, 如果 $c_{i,j} > f_{i,j}$, 构造边 $e_{i,j}^1$, 容量为 $c_{i,j} - f_{i,j}$, 单位费用为 $d_{i,j}$;
对于所有边 $e_{i,j}$, 如果 $f_{i,j} > 0$, 构造边 $e_{j,i}^2$, 容量为 $f_{i,j}$, 单位费用为 $-d_{i,j}$ 。
- K2: 在补图上找负价环 C^- 。若无负价环, 算法终止。



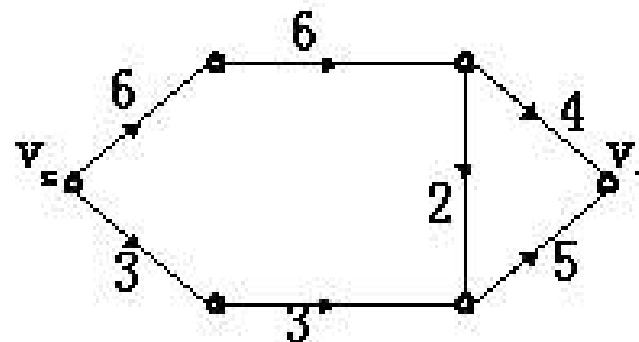
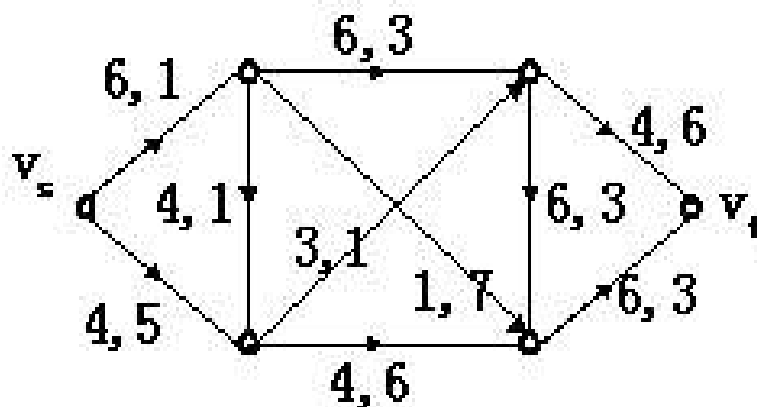
5.3.3 最小费用流问题

- K3: 在负价环上沿环方向使各边增流, 增流数为:
$$\delta = \min_{(i,j) \in C^-} (c_{i,j}^*)$$
- K4: 修改原图每边的流量, 得新可行流。
- K5: 返回K1。
- 上面的算法中需要寻找负价环, 这个可以通过Floyd算法来解决。



5.3.3 最小费用流问题

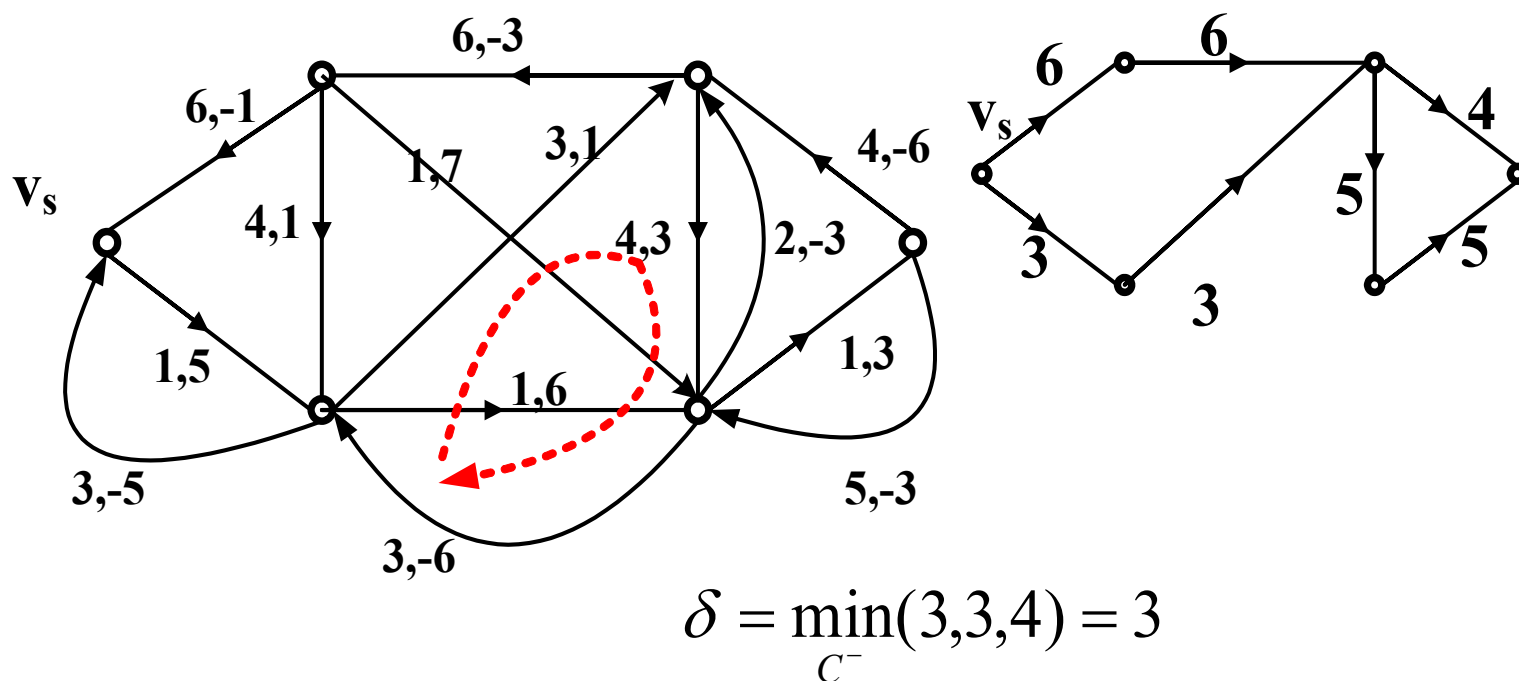
例5.10：已知 $c_{i,j}$, $d_{i,j}$, 要求 $F = 9$ 。
求最小费用流。





5.3.3 最小费用流问题

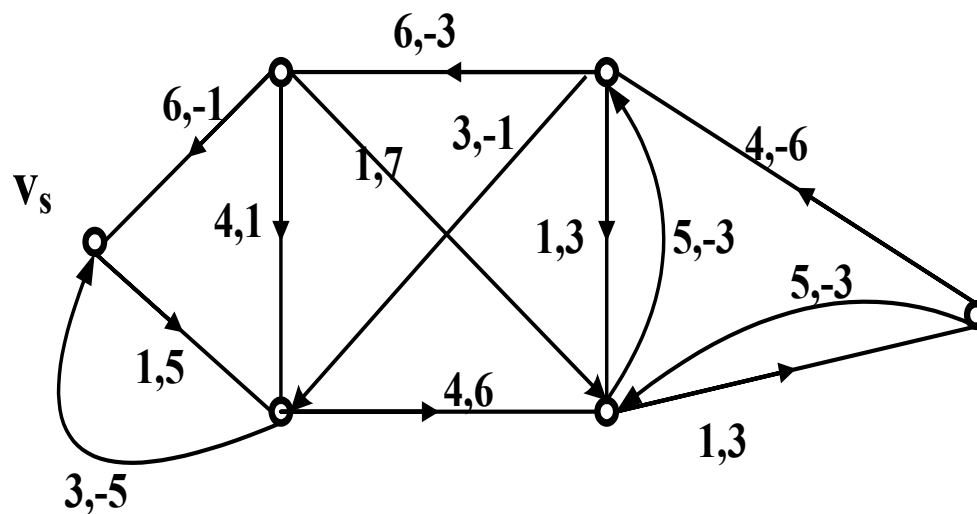
解：上面可行流安排总费用为： $C=102$ 。下面做补图，寻找负价环，调整可行流。





5.3.3 最小费用流问题

- 这个新可行流的补图如下：这个补图没有负价环。



习题



- 5.4
- 5.6
- 5.8
- 5.10